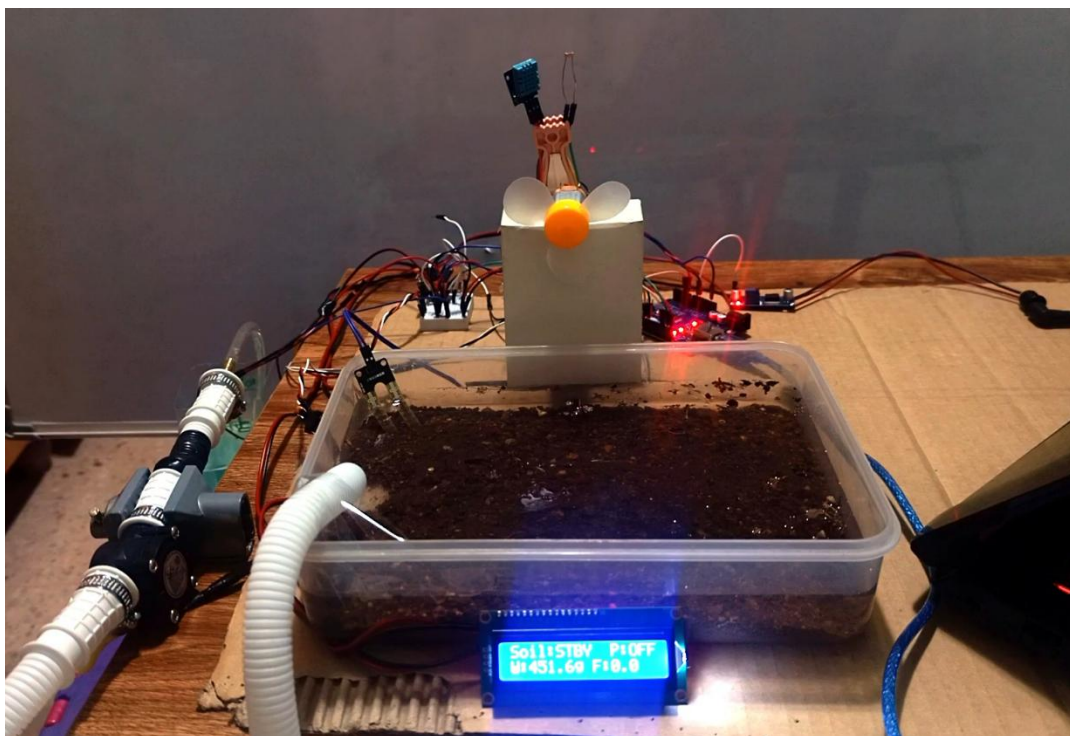




Sensors Project Final Report

Smart Greenhouse



Presented to: Dr. Mohamad Aoude

Prepared by: Group A4

- Mostafa Shaaban 6767
- Ali Daher 6927
- Ghassan Iskandarani 6928
- Hassan Hareb 6777

3/7/2026

Table of Contents

Page 3 - Introduction

Page 5 - Problem Statement

Page 6 - System Architecture

Page 9 - Sensor Selection and Justification

Page 12 - Hardware Design and Schematics

Page 15 - Software Design and Firmware Architecture

Page 20 - Calibration Procedures and Results

Page 28 - Experimental Results and Data Analysis

Page 35 - Measurement Uncertainty and Error
Analysis

Page 40 - Engineering Constraints Discussion

Page 42 - Conclusion and Future Work

Page 44 - References

I. Introduction

In modern engineering, the transition from traditional agriculture to intelligent automation relies entirely on our ability to monitor physical environmental changes in real time. Sensors serve as the critical foundation of this transition, acting as the primary sensory organs that convert physical variables into precise electrical data for a microcontroller to process. Within a Smart Greenhouse, this continuous feedback loop sensing, processing, and actuating transforms a passive shelter into an intelligent, self-regulating ecosystem. Without precise data collection from an integrated sensor network, dynamic automated control would be impossible, leaving delicate crops vulnerable to unpredictable microclimate shifts and inefficient resource management.



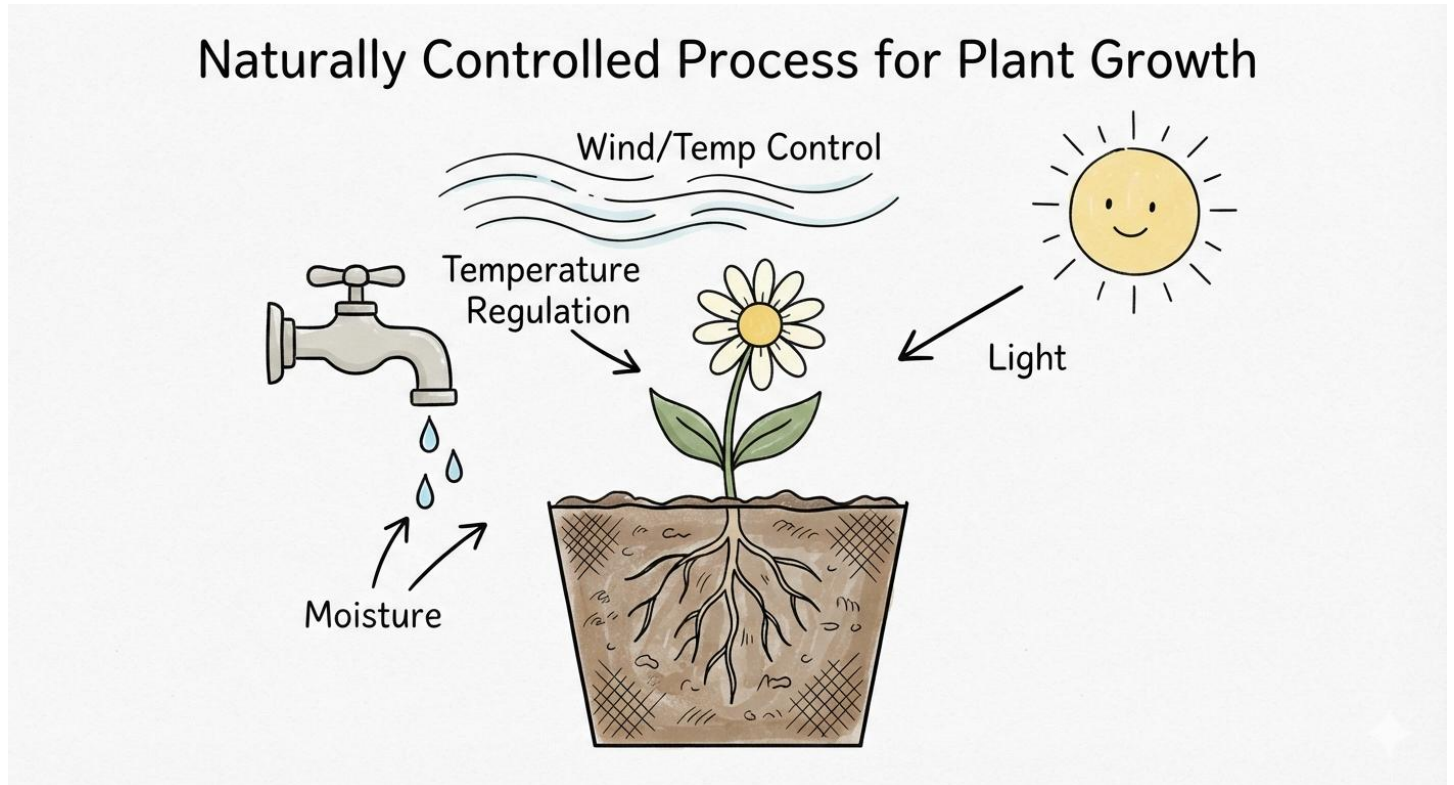
To achieve complete greenhouse homeostasis, this project explores a diverse array of sensor applications, where each transducer serves a distinct operational purpose. A temperature sensor and a humidity sensor continuously monitor the ambient air to



prevent thermal shock and mold growth, while a Light Dependent Resistor (LDR) tracks solar intensity to regulate automated shading or supplemental lighting. Below the surface, a soil moisture sensor gauges the volumetric water content of the earth, signaling a high-power fluid pump to initiate irrigation. To manage the physical resources of the system, a force sensor is deployed underneath the main water reservoir to track the weight of the tank, providing a real-time measurement of fluid depletion and supply levels. Finally, a flow sensor measures the actual water volume passing through the lines during operation to ensure the pump executes the physical delivery safely, preventing line bursts or dry running. By anchoring the entire control loop to these specialized sensing nodes, the system ensures optimal crop growth while maximizing water and energy efficiency.

2. Problem Statement

2.1 Introduction and Background



As we all know, a plant's survival depends on several factors, it requires the right amount of sunlight to undergo photosynthesis, and the right amount of moisture in soil to keep its roots healthy, and wind as a temperature regulator to stay healthy overall.

However, in real environment, depending on nature to provide these requirements is not a guarantee for the health of the plant due to natural variability.

2.2 The Core Problem

One issue might rise from manually watering and nourishing the plant, is using excessive resources such as water, thus causing losses, and excessive sudden temperature spikes, making the usual greenhouse care inadequate for an efficient care of the crops.

2.3 Purpose of the Project

The objective of this project is to design and implement an automated, closed-loop microclimate control system. This system aims to precisely regulate soil moisture, ambient temperature, and light exposure, ensuring optimal plant health while strictly minimizing the waste of water and electrical resources.

3. System Architecture

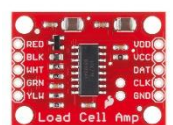
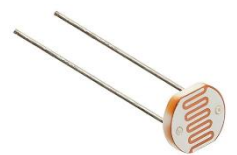
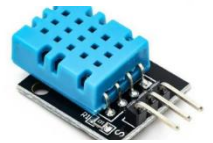
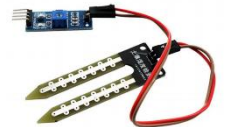
3.1 High-Level Overview

The automated greenhouse control system operates as a closed-loop feedback mechanism. The architecture is divided into three primary subsystems: the input sensing layer, the central processing unit, and the output actuation layer. The system continuously monitors environmental variables and dynamically adjusts physical actuators to maintain predefined operational thresholds.

3.2 Input Subsystem (Data Acquisition)

This layer consists of the sensors responsible for gathering real-time environmental data:

- **Soil Moisture Sensor (HW-080):** Continuously measures the volumetric water content in the soil.
- **Temperature Sensor (DHT11):** Monitors the ambient air temperature directly above the soil bed.
- **Photoresistor (12mm LDR):** Measures the intensity of incoming solar exposure to track light availability.
- **Flow Meter (YF-S201):** Positioned in the irrigation line to measure the volume of water dispensed by the pump in liters per minute (L/min).
- **HX711 module + 5kg load cell.**



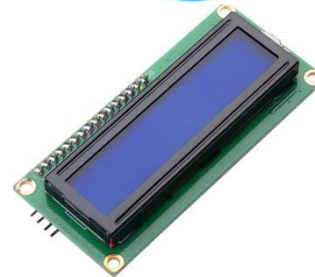
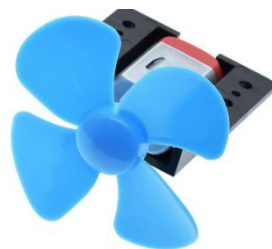
3.3 Central Processing Unit (Control Logic)

We used an Arduino Uno as the central processing hub. It reads analog and digital signals from the input subsystem, processes the data through a programmed logic loop (written in C++), and makes real time decisions based on hardcoded environmental thresholds. Additionally, it communicates via the I2C protocol to transmit real-time system states to the local display.

3.4 Actuation Subsystem (Outputs)

This layer executes the physical changes commanded by the microcontroller to restore equilibrium to the microclimate:

- **Irrigation Pump (3-5V DC):** Actuated via a relay module when soil moisture drops below the acceptable threshold, delivering water directly to the soil.
- **Cooling Fan (3-6V DC motor):** Activated to initiate forced convection cooling when the ambient air temperature exceeds the upper safety limit.
- **16x2 I2C LCD Screen:** Serves as the local user interface, displaying real-time diagnostic data including current temperature, soil moisture levels, and active water flow rates (L/min) for immediate on-site monitoring.



3.5 Signal Flow and System Logic

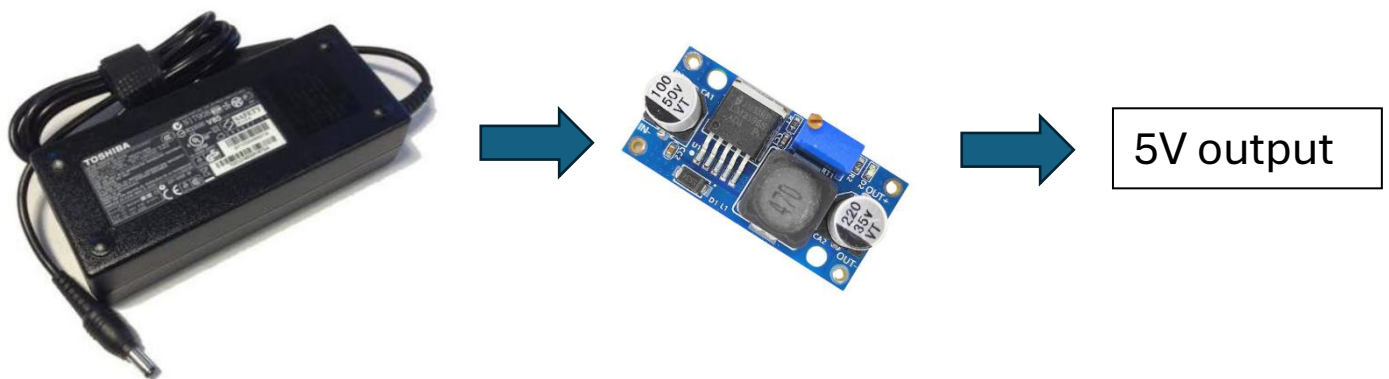
The control loop operates continuously. If the temperature sensor registers a value above the maximum setpoint T_{max} , the microcontroller outputs a signal to engage the cooling fan.

Simultaneously, if the soil moisture sensor detects low humidity, the microcontroller triggers the water pump. The flow meter provides secondary feedback by logging the exact volume of water dispensed,

ensuring resource efficiency. Once sensors confirm that parameters have returned to the optimal range, the microcontroller deactivates the respective actuators.

3.6 Power Distribution Subsystem

The power is derived from a 19V, 6.3A primary supply adapter. To accommodate the logic-level components, an LM2596 step-down buck converter regulates this input to a stable 5V rail. This dedicated rail powers the Pump and the fan's motor, isolating the logic circuitry from voltage fluctuations



Also, the Arduino Uno and all of the sensors are powered from the laptop's USB port, and we used MOSFETs instead of relays for the pump and the fan.

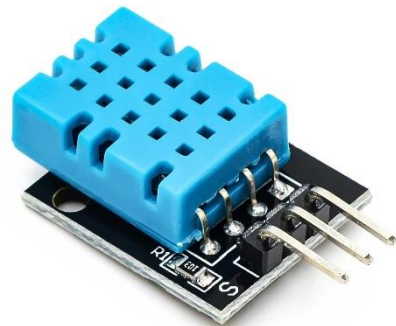
4. Sensor Selection and Justification

To achieve precise, automated homeostasis within the Smart Greenhouse ecosystem, selecting the appropriate transducers is a critical engineering decision. Each component has been selected based on its operating principles, voltage compatibility with the Arduino Uno microcontroller, and its specific functional role within the agricultural control loop.

4.1 DHT11 Digital Temperature and Humidity Sensor:

Operating Principle: The DHT11 utilizes a capacitive humidity sensing element and a high-precision thermistor to measure the surrounding air, featuring an onboard 8-bit microcontroller that outputs a pre-calibrated digital signal.

Justification: Relative humidity and ambient temperature govern crop transpiration and protect against fungal pathogens. While older analog sensors are vulnerable to electrical noise over long wire runs, the DHT11 transmits data via a single-wire digital bus. This ensures complete signal integrity even when placed near high-current motor lines.



4.2 Light Dependent Resistor (LDR):

Operating Principle: An LDR is a photoconductive cell whose internal electrical resistance drops non-linearly as ambient light intensity increases.

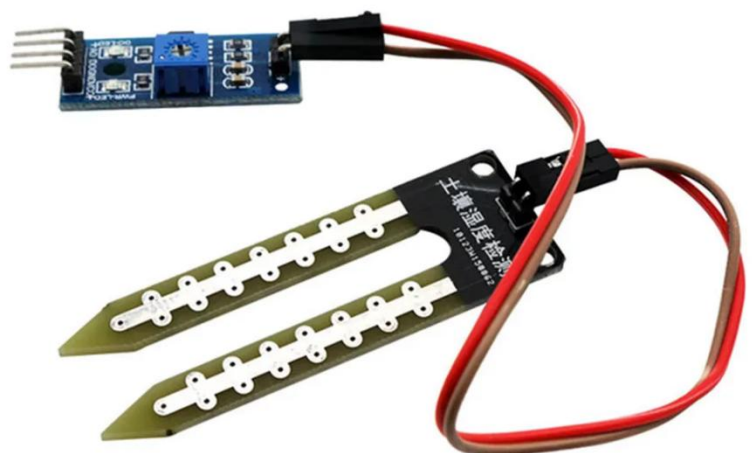
Justification: Solar radiation dictates photosynthetic activity and internal heat accumulation via the greenhouse effect. By placing the LDR in a voltage divider network with a $10\text{ k}\Omega$ pull-down resistor (R5), the changing resistance is converted into a readable voltage string. This allows the microcontroller to monitor daylight availability.



4.3 HW-080 Soil Hygrometer (Moisture Sensor):

Operating Principle: Deployed directly into the crop substrate, the HW-080 probe passes a small current between two exposed pads to measure the soil's electrical resistance, which varies inversely with the volumetric water content.

Justification: Soil moisture levels dictate irrigation timing. The probe passes its analog signal to the HW-080 comparator board, which outputs a continuous analog voltage. This feedback loop eliminates open-loop timer-based watering, ensuring the irrigation pump triggers only when the root zone encounters genuine moisture depletion.



4.4 Load Cell with HX711 Amplifier Module (Tank Weight):

Operating Principle: The load cell contains internal strain gauges arranged in a Wheatstone bridge configuration that change resistance when physical weight deforms the metal bar. Because these electrical changes are microvolt-scale, the HX711 module acts as a high-precision 24-bit Analog-to-Digital Converter (ADC) to amplify and digitize the signal.

Justification: Tracking resource levels is a vital constraint in automation. Positioned underneath the main water reservoir, this assembly tracks the physical weight of the tank. The HX711 communicates via a 2-wire digital serial interface (Data and Clock), giving the Arduino real-time data to prevent the water pump from running dry when the reservoir is depleted.



4.5 YF-S201 Hall-Effect Fluid Flow Meter:

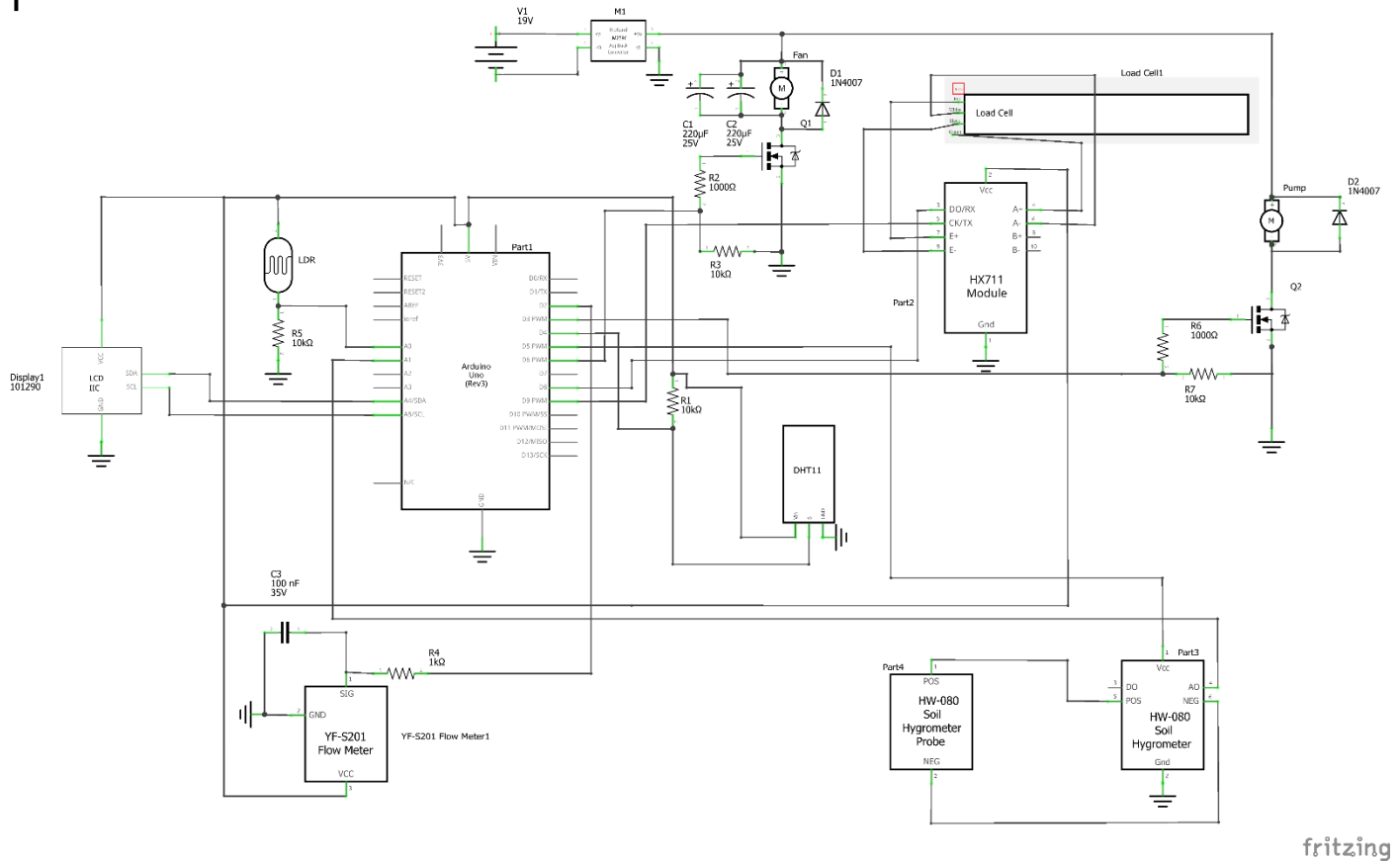
Operating Principle: This inline sensor features an internal plastic pinwheel embedded with a permanent magnet. As water flows through it, the pinwheel spins past a stationary Hall-effect semiconductor chip, which generates a digital square-wave pulse train proportional to the fluid velocity.



Justification: While the soil moisture probe indicates when to water, the flow meter tracks the exact volume of water delivered. This serves as a vital diagnostic layer: if the pump is switched on but the flow rate is 0 L/min, the system immediately flags a physical line blockage or mechanical pump failure.

5. Hardware Design and Schematic

Here's the project's wiring schematic, produced in Fritzing:



5.1 Powering the greenhouse:

The system is powered from a 19V 6.3A laptop charger (V1), stepped down to 5V 3A by an LM2596 adjustable buck converter (M1) to supply the actuator subsystems. Stepping down from 19V rather than supplying regulated 5V directly allows headroom for the pump (3-5V) and fan (3-6V DC motor). Two 220 μ F/25V electrolytic capacitors (C1, C2) are placed at the fan supply node to buffer the rail against current transients caused by PWM switching, reducing supply-borne noise that could otherwise couple into nearby signal lines.

As for the Arduino Uno, it's powered by the laptop's 5V USB port which gives us the ability to serial monitor the outputs of all the sensors. Every sensor in the greenhouse is being powered using the Arduino's 5V pin.

5.2 Actuator Driver Stage:

Both the fan and the irrigation pump are switched using IRLZ44N logic-level N-channel MOSFETs (Q1 for the fan, Q2 for the pump). The IRLZ44N was selected specifically because it is a logic-level device it reaches full conduction with a 5V gate drive directly from the Arduino's digital output pins, eliminating the need for a separate gate driver IC.

Each MOSFET's gate is driven through a $1\text{k}\Omega$ series resistor (R2 for the fan, R6 for the pump) to limit inrush current into the gate capacitance during switching, and a $10\text{k}\Omega$ pulldown resistor (R3, R7) holds the gate at ground when the Arduino pin is high-impedance, preventing the MOSFET from turning on unpredictably.

Flyback diodes (D1, D2 - 1N4007) are placed across the fan and pump motor terminals to clamp the inductive voltage spike generated when current through the fan and pump winding is interrupted, protecting the MOSFETs from avalanche breakdown.

5.3 Sensing Subsystem:

- **Weight sensing:** A 5kg load cell is wired in a standard four-wire Wheatstone bridge configuration (A+/A-/E+/E-) to an HX711 24-bit ADC module, which communicates with the Arduino over a two-wire digital interface (DOUT/SCK).
- **Flow sensing:** A YF-S201 hall-effect flow sensor outputs a digital pulse train proportional to flow rate. Its signal line passes through an RC low-pass filter ($R4 = 1\text{k}\Omega$, $C3 = 100\text{nF}$) before reaching the Arduino's interrupt pin. This filter attenuates high-frequency switching noise, particularly noise coupled from the nearby pump MOSFET's PWM switching, before it reaches the digital input, providing a first line of defense against false pulse counts at the

hardware layer, complementing the software-level debounce and median filtering discussed in Section 6.

- **Environmental sensing:** A DHT11 provides temperature and humidity via a single-wire digital interface, with a 10kΩ pull-up resistor (R1) on the data line per the manufacturer's datasheet recommendation. Ambient light is measured via a photoresistor (LDR) in a voltage-divider configuration with a 10kΩ fixed resistor (R5), producing an analog voltage proportional to illuminance.
- **Soil moisture:** An HW-080 soil moisture module with a separate probe reports moisture via analog output. The module's power is switched through a dedicated Arduino digital output rather than a permanent supply rail, allowing the firmware to power-cycle the probe only during measurement windows, this mitigates electrolytic corrosion of the probe electrodes from sustained DC exposure, and is discussed further as a software-hardware co-design decision in Section 6.
- **Display:** A 16x2 character LCD with an I2C backpack communicates over the Arduino's SDA/SCL bus, minimizing pin usage compared to a parallel interface.

5.4 Full Pinout Reference:

Signal	Arduino Pin	Interface Type
YF-S201	D2	Digital interrupt
DHT11 data	D4	Digital (1-wire)
Pump MOSFET gate	D3 (PWM)	Digital output
Fan MOSFET gate	D6 (PWM)	Digital output
HX711 DOUT	D8	Digital
HX711 SCK	D9	Digital
LDR divider	A0	Analog input
Soil sensor signal	A1	Analog input
Soil sensor power	D5	Digital output
LCD	A4 (SDA) / A5 (SCL)	I2C

6. Software Design and Firmware Architecture

6.1 Scheduling Model:

The firmware avoids `delay()` in favor of a `millis()` based cooperative scheduling pattern, allowing multiple subsystems to run at independent intervals without blocking one another. Three timing domains coexist within `loop()`:

- Main sensor/control tick (`loopTickInterval = 1000 ms`), sensor sampling, control logic, display update, and logging
- Pump ramp step (`rampStepInterval = 15 ms`), smooth PWM transitions, decoupled from the main tick
- Soil sensor read window (`soilReadInterval = 900,000 ms / 15 min`, `soilReadVWindow = 2,500 ms`), the probe is powered only briefly before each reading

This structure allows the system to service incoming serial commands, ramp the pump, and read sensors within the same loop iteration without any single operation stalling the others.

6.2 Flow Sensor Signal Processing:

Pulses from the YF-S201 are captured via a hardware interrupt (`flowPulseISR`, `RISING` edge) rather than polling, since flow pulses can occur faster than the main loop iterates. The ISR includes a software debounce, rejecting any edge arriving less than `minPulseGapMicros` after the previous accepted pulse, which suppresses high-frequency electrical noise that could otherwise be miscounted as flow pulses,

complementing the RC low-pass filter on the same signal line at the hardware level (Section 5).

Each main tick, the accumulated pulse count is atomically read and reset inside a `noInterrupts()/interrupts()` critical section to prevent a race condition between the ISR and the main loop. The resulting flow rate is then smoothed with a 3-sample median filter (`medianOf()`), which rejects single-sample outliers without the lag a moving-average filter would introduce.

6.3 Sensor Multiplexing:

The DHT11 and HX711 are read on alternating ticks (`alternateStateToggle`) rather than every tick, spreading their respective read latencies across two seconds instead of stacking both within one and keeping the main loop responsive.

The HX711 read is implemented with a bounded-wait strategy (`readScaleMedianSafe()`): rather than blocking indefinitely on `scale.read()`, each of up to `SCALE_SAMPLE_COUNT` samples is given a maximum `SCALE_SAMPLE_TIMEOUT_MS` to become ready before the function proceeds with whatever samples it has collected. This bounds the worst-case blocking time of a weight read to a known ceiling, well within the 1-second tick budget.

Sampling rate per sensor:

- YF-S201:
 - Continuous pulse capture (interrupt-driven), aggregated every 1s
 - Flow pulses can arrive faster than a polled loop could catch; interrupt capture avoids missed pulses. A 1s aggregation window matches the sensor's practical response time and

avoids over-resolving a signal that fluctuates on the order of seconds

- DHT11:
 - Once every 2s (alternating tick)
 - The DHT11 datasheet specifies a minimum ~ 1 s interval between reads; sampling every 2s stays comfortably within that limit while matching how slowly ambient temperature/humidity actually change
- HX711:
 - Once every 2s (alternating tick), oversampled internally (up to 7 raw conversions per reading, median-filtered)
 - Reservoir weight changes slowly except during active irrigation, so a 2s update rate is sufficient; internal oversampling per reading improves noise rejection without needing a faster update rate
- LDR:
 - Once per second (main tick)
 - Ambient light changes gradually outside of fast transients so 1 Hz is more than sufficient to track meaningful change without generating excess data
- HW-080:
 - Once per 15-minute window, sensor powered only ~ 2.5 s per reading
 - Soil moisture changes on the order of minutes to hours, not seconds, so infrequent sampling is physically appropriate; more importantly, the sensor is power-gated to only the measurement window specifically to limit electrode exposure time and reduce electrolytic corrosion of the probe

6.4 Control Logic:

Irrigation control operates as a two-state automaton (idle → running → idle) gated by soil moisture and reservoir weight. The pump activates when soil moisture falls below 25% and reservoir weight is at or above 278.3 g, preventing the system from attempting to irrigate when the reservoir is too low. Once active, the pump runs at a fixed PWM duty target (`targetPwm3 = 180`) and deactivates under either of two independent conditions, whichever occurs first:

- A fixed maximum runtime of 12 seconds elapses, or
- Reservoir weight drops below the same 278.3 g threshold during the run

The runtime cap prevents over-irrigation in the event that the weight-based cutoff fails to trigger, while the weight-based cutoff independently protects against reservoir depletion mid-cycle regardless of elapsed time. Using two independent shutoff conditions rather than one is a safety-oriented design choice, it ensures the pump cannot run unbounded if either individual condition fails to be evaluated correctly.

Thermal control is a threshold-based rule evaluated each tick against the latest DHT11 reading: the fan switches fully on (`pwm6Value = 255`) when temperature exceeds 30°C, and off (`pwm6Value = 0`) at or below 30°C.

PWM ramping is applied uniformly to the pump in both directions, stepping `currentPwm3` toward `targetPwm3` by ± 5 every 15 ms regardless of whether the target is above or below the current value. This symmetric ramping avoids introducing a sharp current step on either pump start or stop, which is relevant given the system's sensitivity to switching-induced electrical noise discussed in Section 5.

6.5 Manual Override Interface:

A serial command interface allows manual control of the pump (P3:<0-255> / P3:AUTO), fan (P6:<0-255> / P6:AUTO), and LCD page (LCD:FIX:<0|1> / LCD:AUTO), each independently switchable between automatic and manual modes via dedicated override flags. This interface supports bench testing, calibration, and controlled triggering of specific system behaviors.

6.6 Data Logging:

Every main tick appends one CSV-formatted row to the serial output, containing a running log index, elapsed time, sensor readings, and actuator states, providing the dataset analyzed in Section 8. NaN sensor readings (e.g., a failed DHT11 checksum) are substituted with a -999 sentinel via logSafe() before logging, since a literal nan value would break CSV parsing in downstream analysis tools.

7. Calibration Procedures and Results

Sensor calibration is a fundamental procedure in data acquisition engineering used to eliminate systematic errors, offset biases, and scaling drifts. By comparing raw sensor readouts against a certified reference instrument (the "Real" value) across a known operational range, a correction mathematical function can be derived to ensure absolute measurement accuracy.

7.1 Temperature Sensor Calibration (DHT11):

7.1.1 Methodology and Data Collection

To calibrate the temperature sensing element of the DHT module, static environment testing was conducted at three distinct reference thermal points. At each point, the raw output of the sensor under test was recorded alongside the measurements from a high-precision calibrated reference thermometer.

The experimental data collected before calibration revealed a distinct systematic positive bias across the operational range:

Point 1 (Low Thermal Range): A reference temperature of 8°C yielded a raw sensor readout of 10.5°C representing an absolute error of $+2.5^{\circ}\text{C}$.

Point 2 (Ambient Room Range): A reference temperature of 26.7°C yielded a raw readout of 27.6°C (absolute error of $+0.9^{\circ}\text{C}$)

Point 3 (Elevated Range): A reference temperature of 26°C yielded a raw readout of 26.2°C (absolute error of $+0.2^{\circ}\text{C}$).

7.1.2 Linear Regression via Microsoft Excel

To mathematically model and eliminate this non-uniform offset, the experimental dataset was exported to Microsoft Excel. Rather than using manual algebraic computations, Excel's built-in statistical engine was utilized to plot the scatter diagram, execute a least-squares linear regression analysis, and generate a dynamic trendline equation.

The automated regression analysis in Excel yielded the following precise calibration transfer function:

$$y = 1.1155x - 3.6764$$

Where:

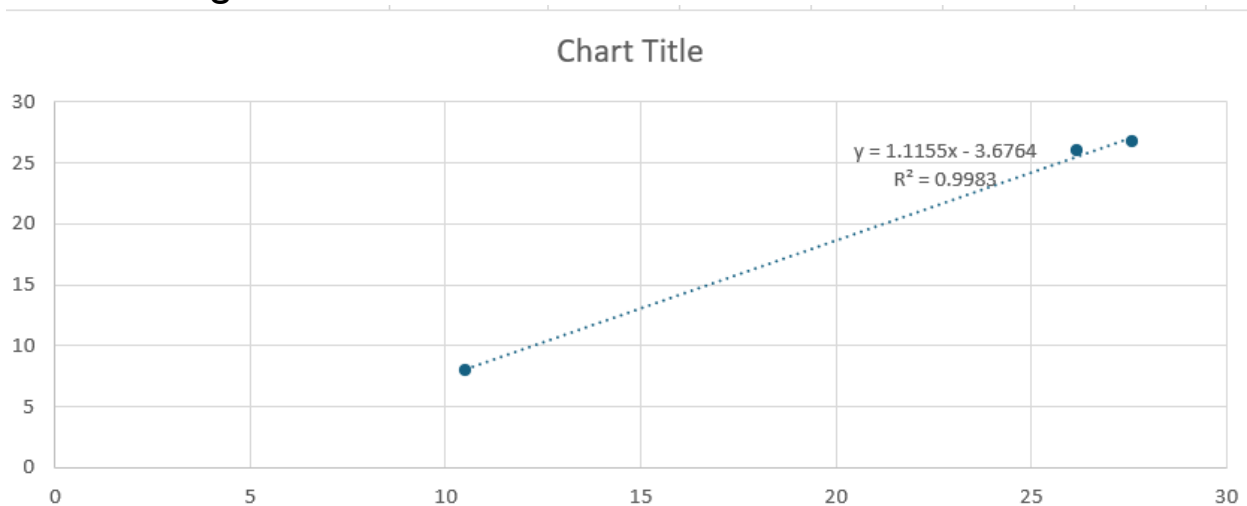
y represents the true, corrected temperature output ($^{\circ}\text{C}$).

x represents the uncalibrated raw value read directly by the microcontroller from the DHT sensor.

Along with the equation, Excel computed the coefficient of determination:

$$R^2 = 0.9983$$

An R^2 value so close to 1 indicates an exceptionally high linear correlation, confirming that the sensor's raw behavior is highly predictable and structurally correctable using Excel's derived linear model.



7.1.3 Validation Post-Calibration

To validate the efficacy of this correction factor, the Excel-derived regression formula was embedded directly into the Arduino firmware logic. A validation test run was executed at a real ambient reference temperature of 26.15°C .

Passing the uncalibrated raw sensor input through the transfer function yielded a post-calibration system value of 26.35°C . This reduced the absolute operational error down to an acceptable margin of only 0.2°C (a minimal relative error of 0.765%), successfully bringing the sensor within strict engineering tolerances for automated greenhouse climate safety.

7.2 Light Dependent Resistor (LDR) Calibration:

7.2.1 Experimental Data Collection

To map raw sensor readings to actual environmental light levels, experimental testing was conducted across four distinct illumination states. The raw 10-bit ADC values read by the Arduino from the voltage divider circuit were plotted against physical light levels recorded in lux using a reference lux meter:

Point 1 (Low Light): A raw ADC value of 341 corresponds to a reference level of 10 lux.

Point 2 (Overcast/Shade): A raw ADC value of 600 corresponds to a reference level of 90 lux.

Point 3 (Indoor/Ambient): A raw ADC value of 626 corresponds to a reference level of 190 lux.

Point 4 (Direct Exposure): A raw ADC value of 900 corresponds to a reference level of 2400 lux.

7.2.2 Non-Linear Mathematical Modeling via Microsoft Excel

Because photoconductive cells naturally display a non-linear resistance drop under photon exposure, an exponential regression model was fitted to the dataset using Microsoft Excel's trendline analysis tools.

The automated regression analysis in Excel generated the following exponential mathematical calibration function:

$$y = 0.3274e^{0.0098x}$$

Where:

y represents the true environmental illumination output (lux).

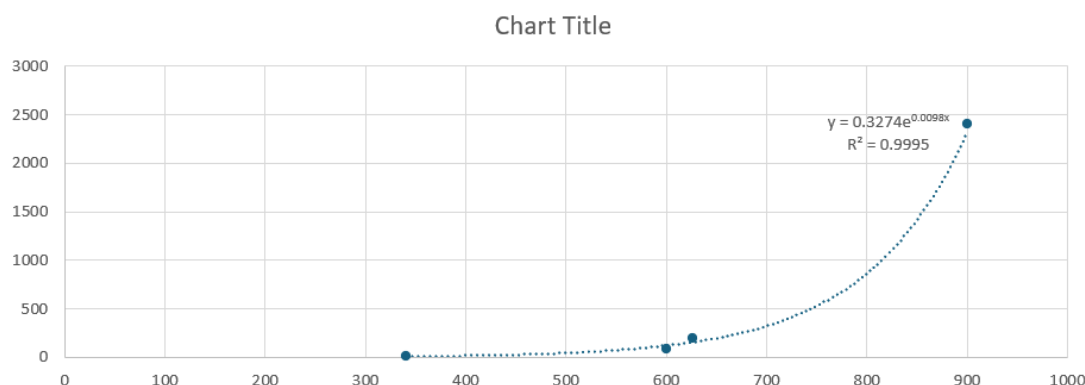
x represents the uncalibrated raw 10-bit ADC integer value (ranging from 0 to 1023) read by the microcontroller.

e is Euler's constant (≈ 2.71828).

Along with the function, Excel computed the coefficient of determination:

$$R^2 = 0.9995$$

An R^2 value of 0.9995 confirms an exceptionally strong statistical fit, verifying that the non-linear behavior of the photoresistor can be reliably linearized in the software.



7.2.3 Validation Post-Calibration

To validate the reliability of this Excel-derived exponential model, a separate verification test run was conducted. Under an independent test environment, a real ambient reference light level of 146.00 lux was established.

When the raw ADC input from the LDR was processed through the firmware equation, the system calculated a corrected output value of 145.33 lux. This yields an absolute operational error of only 0.67 lux (a minimal relative error of 0.459%), confirming that the calibration function provides the precision necessary for automated lux-level control in the greenhouse.

7.3 Soil Moisture Sensor (HW-080) Calibration:

7.3.1 Experimental Data Collection

To establish a direct correlation between raw microelectronic parameters and physical environmental states, a two-point calibration process was conducted to find the absolute boundaries of the substrate's volumetric water content. The 10-bit raw ADC integer readouts from the HW-080 analog interface pin were recorded under complete dryness and maximum water saturation:

Point 1 (Completely Dry Soil / Air Exposure): A physical soil moisture level of 0% wetness yielded a maximum raw ADC integer output of 1006.

Point 2 (Fully Saturated / Mud Substrate): A physical soil moisture level of 100% wetness yielded a minimum raw ADC integer output of 602.5.

7.3.2 Linear Regression via Microsoft Excel

Because resistive soil sensors feature an inverse relationship, where increased electrical conductivity (more water) drops the voltage drop read by the microcontroller, the dataset displays a clear negative slope. The experimental parameters were exported into Microsoft Excel to compute a precise linear trendline.

The automated least-squares regression analysis performed by Excel generated the following calibration transfer function:

$$y = -0.2478x + 249.32$$

Where:

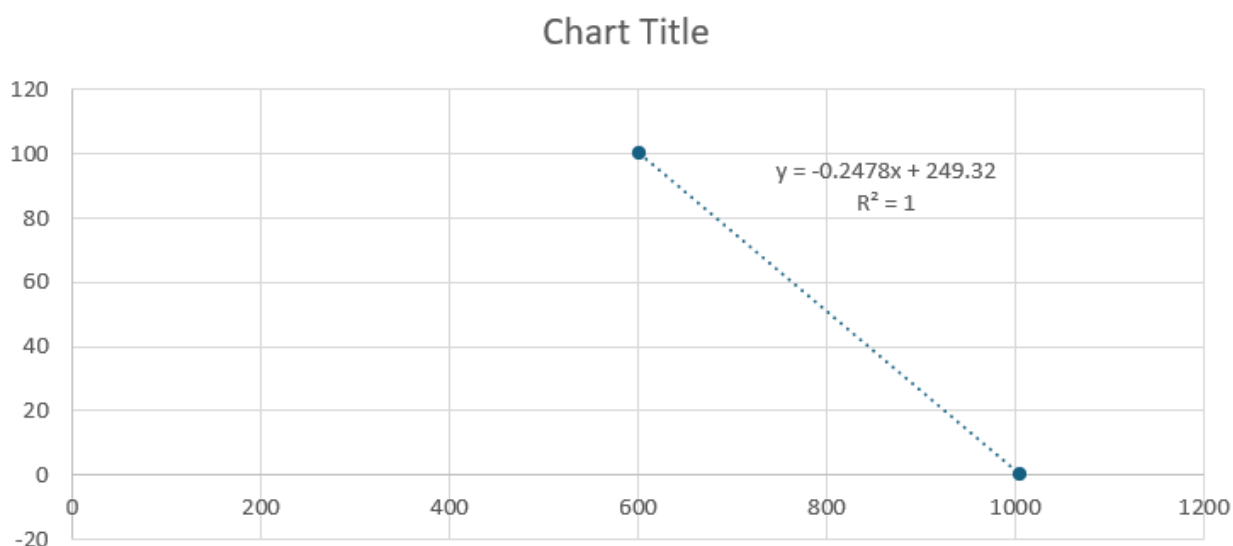
y represents the true, calculated volumetric soil moisture percentage (% wetness).

x represents the uncalibrated raw 10-bit ADC value read on Analog Pin A1 from the HW-080 sensor.

Along with this linear transfer model, Excel generated the coefficient of determination:

$$R^2 = 1$$

An R^2 value of exactly 1 indicates a perfect mathematical fit through the two primary boundary thresholds. This linear equation is easily programmable within the Arduino firmware loop, converting arbitrary raw input values directly into readable 0% to 100% soil hydration metrics to regulate the irrigation pump efficiently.



7.4 Load Cell (HX711) Calibration:

7.4.1 Experimental Data Collection

To calibrate the load cell monitoring the water reservoir tank, static mass testing was conducted across four reference weights. The raw uncalibrated 24-bit digital serial values transmitted by the HX711 ADC module were recorded against known physical mass standards measured in grams (g):

Point 1 (Zero Load Baseline): A real mass of 0 g yielded a raw internal digital reading of -334720.

Point 2 (Low Load): A real mass of 109 g yielded a raw internal digital reading of -291830.

Point 3 (Mid Load): A real mass of 226 g yielded a raw internal digital reading of -238760.

Point 4 (High Load): A real mass of 566 g yielded a raw internal digital reading of -116510.

7.4.2 Linear Regression via Microsoft Excel

Because strain gauges operate on a strictly proportional resistance change under mechanical stress, the dataset demonstrates excellent linearity. The values were processed through Microsoft Excel's built-in data analysis engine to compute a least-squares linear regression trendline.

The automated trendline analysis in Excel generated the following calibration transfer function:

$$y = 0.0026x + 861.47 + \text{offset}$$

Where:

y represents the true calculated mass of the reservoir tank (g).

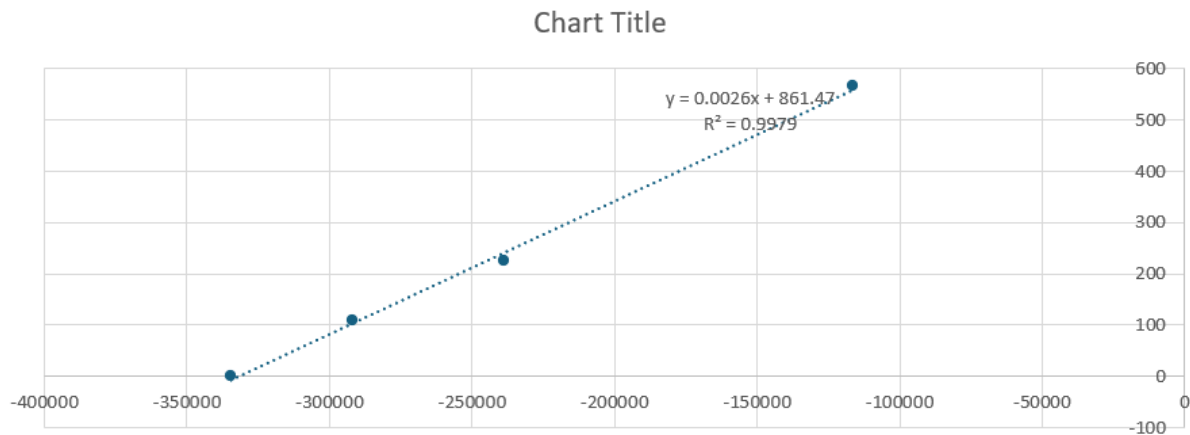
x represents the raw digital integer output from the 24-bit HX711 module interface.

offset is an adjustable parameter accounting for the initial tare weight depending on the physical resting position of the module.

Along with this transfer function, Excel calculated the coefficient of determination:

$$R^2 = 0.9975$$

An R^2 value of 0.9975 confirms an exceptionally high linear correlation, proving that the raw output can be directly converted into accurate structural load units.



7.4.3 Validation Post-Calibration

To verify the accuracy of this linear scaling model, a verification test run was conducted on the physical assembly. A reference mass of 4.90 g was applied to the load cell.

When the raw sensor value passed through the Excel-derived calibration equation, the firmware computed a measured mass output of 4.80 g. This results in an absolute operational error of only 0.10 g (a minimal relative error of 2.04%), providing more than enough resolution to accurately track fluid depletion in the greenhouse supply reservoir.

7.5 Flow Meter (YF-S201) Calibration:

7.5.1 Experimental Data Collection and Factor Derivation

To calibrate the inline Hall-effect fluid flow sensor, volumetric testing was conducted by measuring the total number of square-wave digital pulses generated as a precise volume of fluid passed through the turbine.

The baseline calibration test monitored the cumulative pulse count required to pass three identical successive batches of 0.4 L of water:

Batch 1: 152 pulses

Batch 2: 142 pulses

Batch 3: 165 pulses

Summing these individual test segments yields a total of 459 pulses generated to displace a combined volume of 1.2 L. Minor pulse variations between trials are typical in experimental fluid dynamics due to slight changes in transient pressure and turbine blade inertia. Averaging these runs over the total volume eliminates this random error. Following the instructions outlined in the manufacturer's technical data sheet, the operational Flow Factor (representing the sensor's characteristic pulse frequency per unit volume) was derived mathematically:

$$\text{Flow Factor} = 459 \text{ pulses} / 1.2 \text{ L} = 382.5 \text{ pulses/L}$$

7.5.2 Flow Rate Calculation Formula

With the characteristic Flow Factor derived directly from the experimental data, the continuous real-time volumetric flow rate (L/min) is calculated in the Arduino firmware loop using the following standard data sheet equation:

$$Q = (\text{Pulse Count} / \text{Flow Factor}) * 60$$

Where:

Q represents the calculated real-time flow rate (L/min).

Pulse Count is the number of rising or falling digital pulse edges counted via the hardware interrupt pin over a set sample time interval (typically 1 second).

Flow Factor is the derived metric (382.5 pulses/L).

60.0 is the time conversion factor scaling seconds to minutes.

7.5.3 Validation Post-Calibration

To validate the reliability of this calibrated transfer equation, a fluid flow rate of 1.5 L/min was physically passed through the system as an independent reference test baseline.

When the microcontroller counted the live pulse stream and executed the data sheet formula using the custom 382.5 pulses/L constant, the firmware output matched the reference precisely, registering a measured flow rate of 1.5 L/min. This yields an operational calibration error of 0.00%, confirming that the sensor will accurately monitor real-time fluid delivery rates and instantly capture physical line blockages or dry-pump scenarios.

8. Experimental Results and Data Analysis

To validate the greenhouse controller, the complete system was operated continuously for approximately one hour while all sensor readings and actuator states were logged once per second in CSV format, as described in Section 6. The recorded data were imported into MATLAB for offline analysis and visualization. Five plots were generated showing the environmental conditions, reservoir weight, flow rate, and soil moisture throughout the experiment.

8.1 Environmental Conditions:

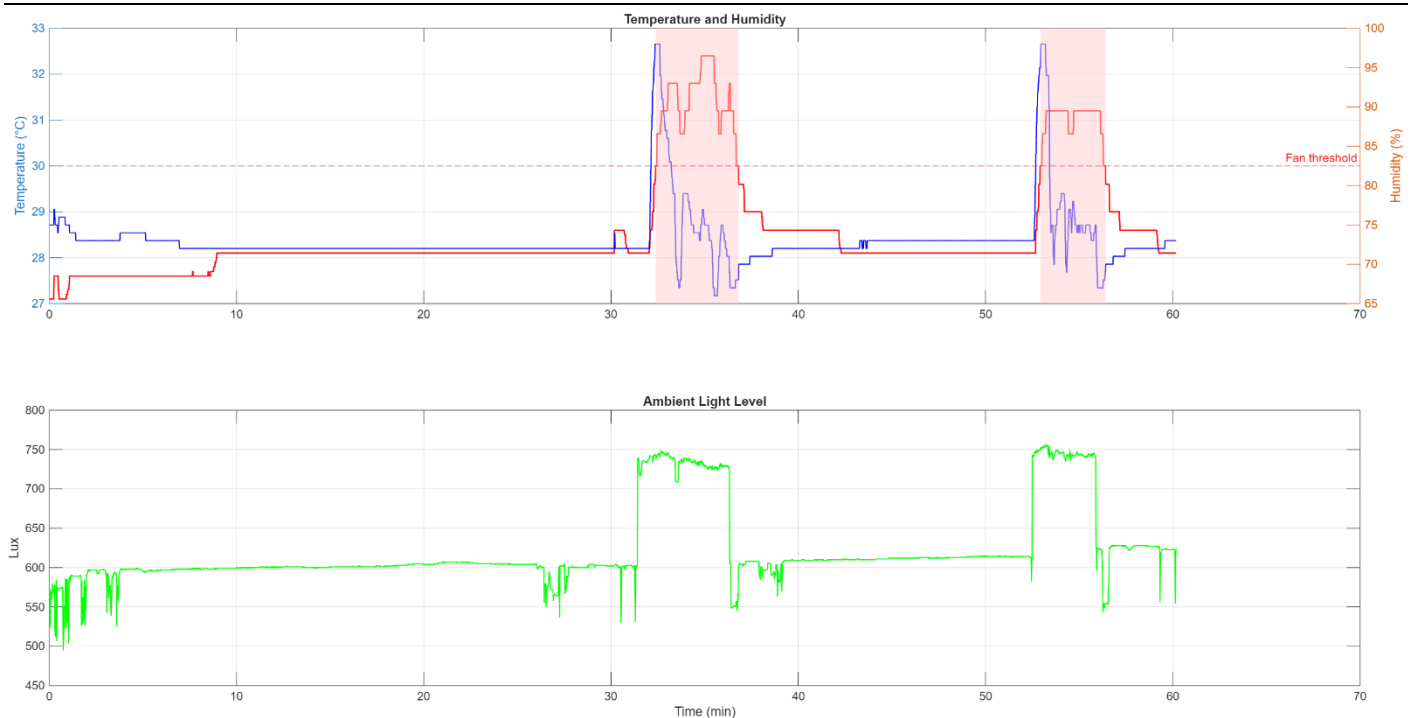


Figure 8.01

Figure 8.01 presents the measured temperature, humidity, and ambient light level over the test duration.

The greenhouse initially operated under stable environmental conditions with temperature remaining between approximately 28–29°C, below the programmed fan activation threshold of 30°C. During

this period the fan remained off, confirming that the thermal controller did not activate unnecessarily.

Two heating events were intentionally introduced at approximately 32 minutes and 53 minutes by exposing the DHT11 sensor to a warm source (thumb). In both cases the measured temperature exceeded 30°C, causing the firmware to immediately command the fan to full speed (PWM = 255), shown by the shaded regions in Figure 8.01. Once the temperature fell below the threshold, the fan automatically switched off.

Humidity was constant at 70% until the thumb touched the DHT11 which increased it to 85-95%.

Ambient light remained relatively constant at approximately 600 lux during most of the experiment. Temporary increases to approximately 740–750 lux when the temperature raised (to simulate a cloud moving and revealing the sun). Aside from these intentional disturbances, the light sensor exhibited good stability with only minor measurement noise.

Overall, the environmental measurements confirm that the sensing subsystem operated continuously throughout the experiment and that the fan control algorithm responded correctly to temperature changes without oscillation or false triggering.

8.2 Irrigation System Performance:

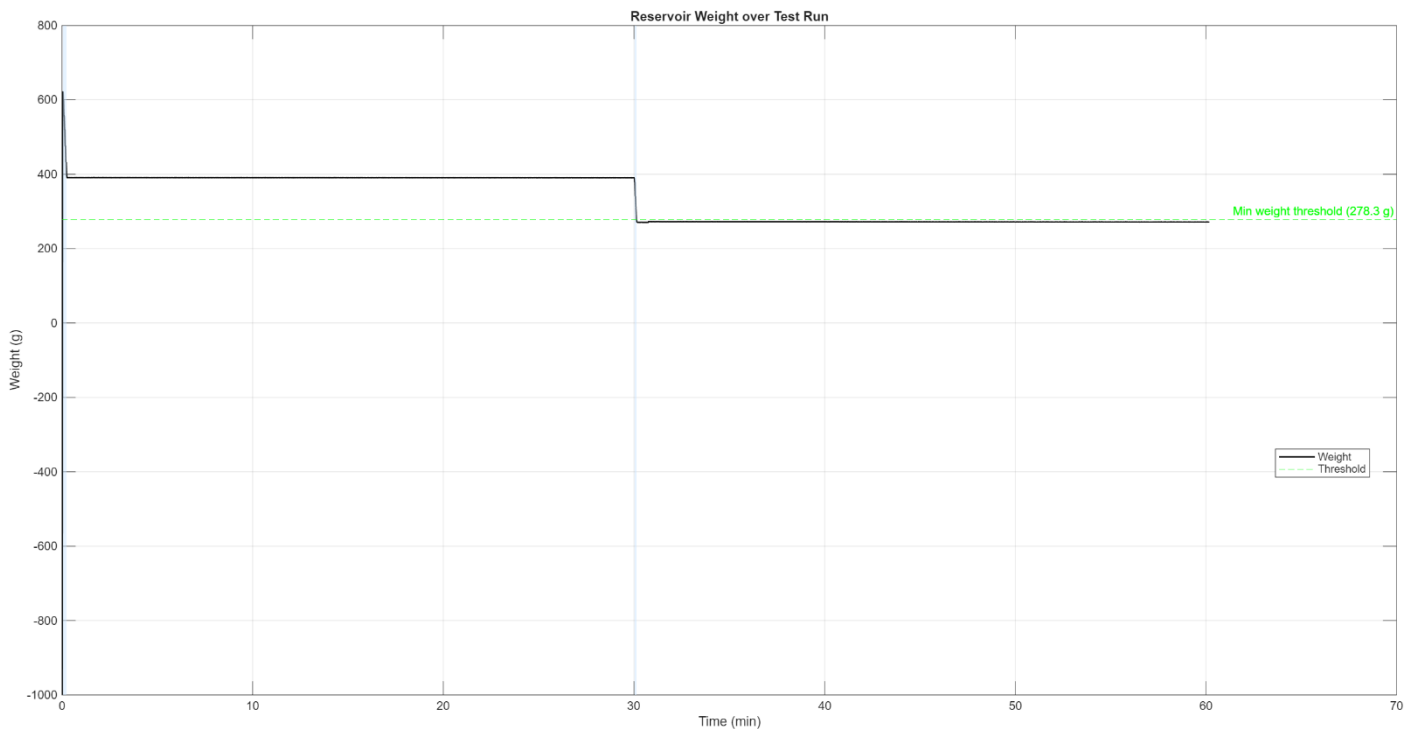


Figure 8.02

Figure 8.02 presents the measured reservoir weight throughout the one-hour experiment together with the programmed minimum reservoir threshold of 278.3 g.

At startup, the load cell measured an initial reservoir weight of approximately 621.4 g. The first soil moisture measurement, taken at $t = 0$ min, was 0%, which is below the irrigation threshold of 25%. Since the reservoir weight was well above the minimum allowable value, the controller immediately initiated an irrigation cycle as specified by the control logic.

The pump operated for the programmed maximum duration of 12 s, after which it automatically stopped. During this interval, the reservoir weight decreased rapidly from approximately 621.4 g to 390.9 g, indicating that approximately $621.4 - 390.9 = 230.5$ g of water was delivered.

The reservoir weight then remained nearly constant until approximately 30.9 min, when the next soil moisture measurement again reported 0%. This triggered a second irrigation cycle. During this

cycle, the reservoir weight decreased from approximately 390.9 g to 272.5 g. Because the reservoir weight fell below the programmed safety threshold of 278.3 g, the controller terminated the pump before any further irrigation could occur.

For the remainder of the experiment, the reservoir weight remained approximately 272 g, and no additional irrigation cycles were initiated despite subsequent soil moisture measurements. This behavior confirms that the low-reservoir protection successfully inhibited further pump operation once the water supply became insufficient.

Overall, the reservoir measurements verify both irrigation shutdown mechanisms implemented in the firmware: the fixed 12-second runtime limit during the initial watering cycle and the minimum reservoir weight cutoff during the second cycle.

8.3 Pump Flow Verification:

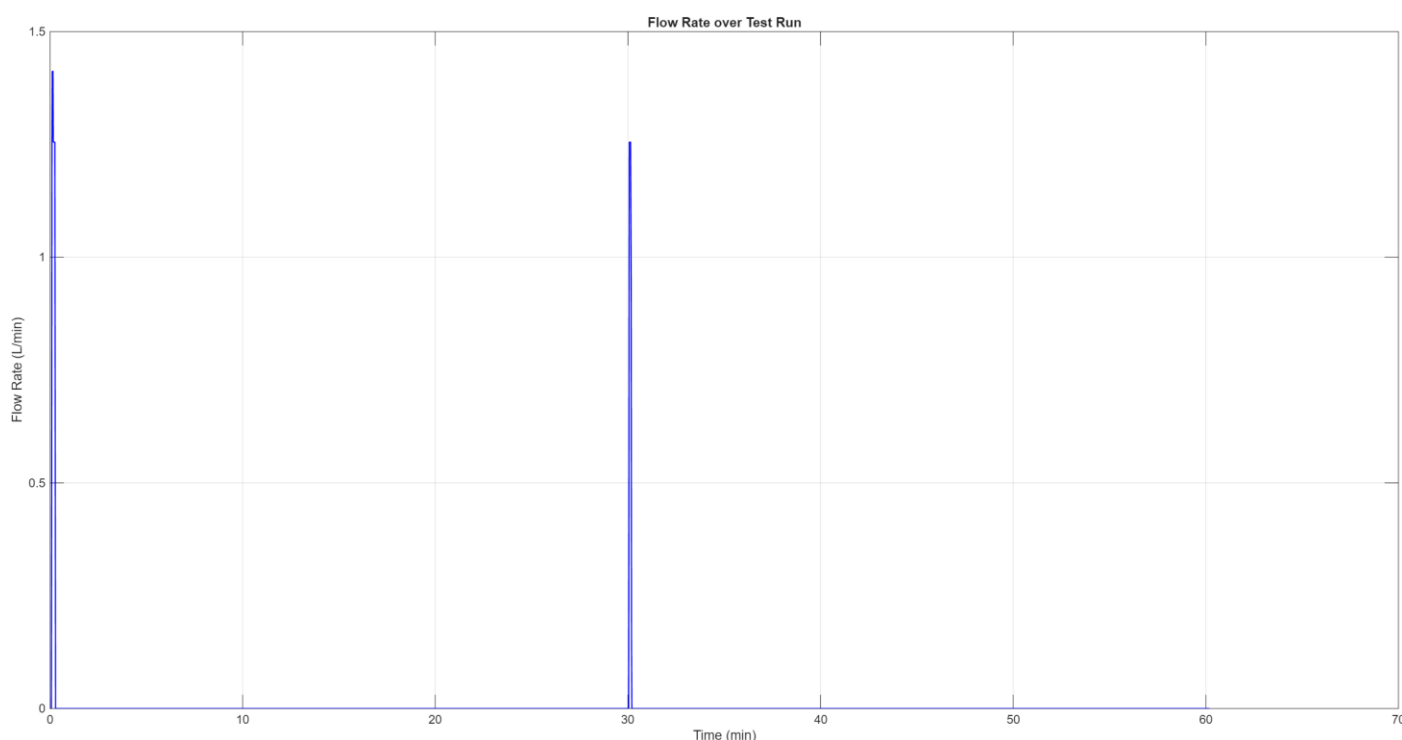


Figure 8.03

Figure 8.03 shows the measured flow rate throughout the experiment. As expected, measurable flow was detected only during active

irrigation cycles, while the recorded flow remained essentially zero for the rest of the one-hour test.

Two distinct flow events are visible. The first occurred immediately after startup when the initial soil moisture reading of 0% triggered the irrigation controller. The second occurred at approximately 30 minutes, when the soil moisture sensor again measured 0%, initiating another irrigation cycle. In both cases, the measured flow rate reached a peak of approximately 1.26 L/min, demonstrating consistent pump performance between successive watering cycles.

The first flow event ended after the firmware's programmed 12-second maximum runtime, confirming correct operation of the time-based shutdown mechanism. During the second irrigation cycle, flow ceased when the reservoir weight fell below the 278.3 g safety threshold, verifying the independent weight-based shutdown implemented in the control algorithm.

No measurable flow was recorded while the pump was inactive, indicating that the flow sensor did not generate false pulses due to electrical noise. This confirms the effectiveness of the signal conditioning approach described in Section 6, including the RC low-pass filter, interrupt-driven pulse counting, software debounce, and median filtering.

The close agreement between the flow measurements, the corresponding decreases in reservoir weight, and the subsequent increase in soil moisture demonstrates that the irrigation subsystem operated as intended. Each pump activation produced a measurable flow event, a corresponding reduction in reservoir mass, and the expected recovery of soil moisture, confirming consistent operation of both the sensing hardware and the irrigation control logic.

8.4 Soil Moisture Response:

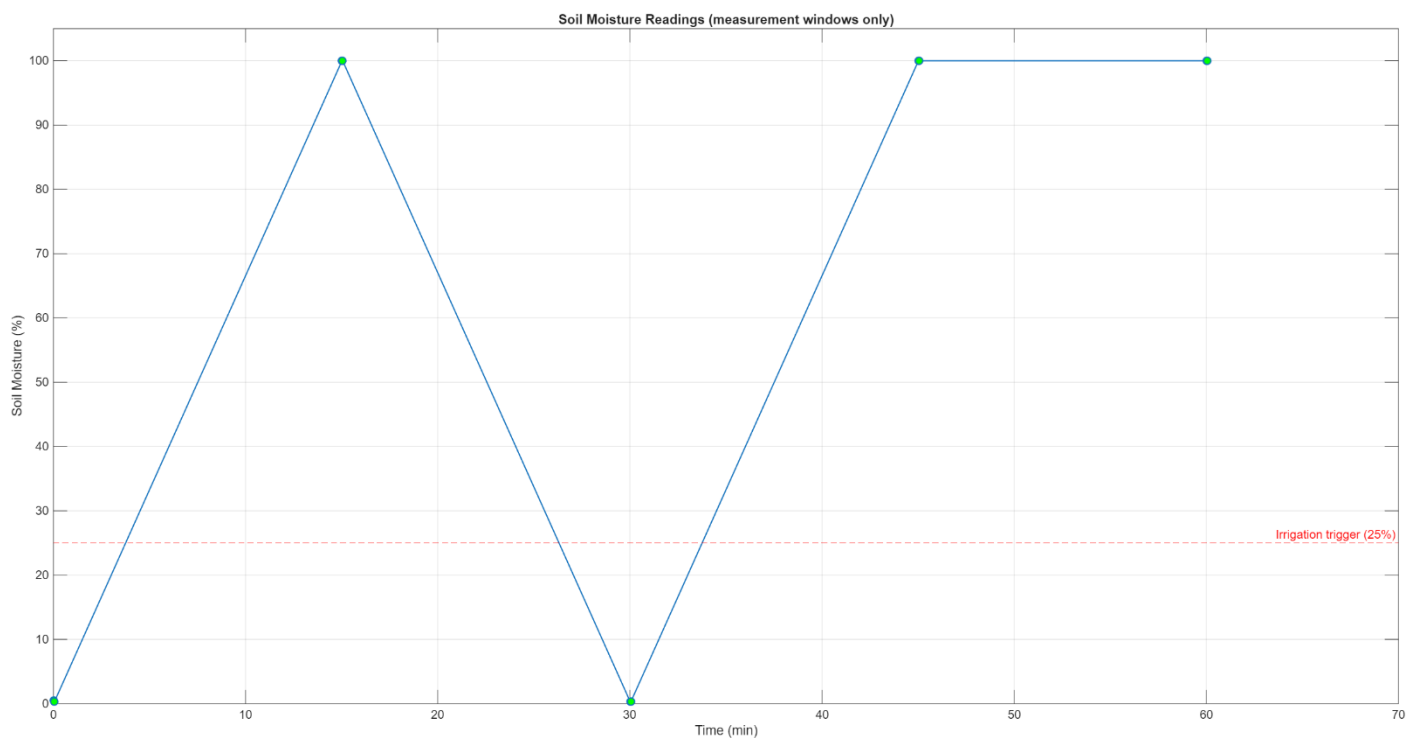


Figure 8.04

The soil moisture sensor was intentionally powered only during short measurement windows occurring every 15 minutes to reduce probe corrosion. Consequently, Figure 8.04 contains only four valid measurements.

The first measurement below the 25% irrigation threshold occurred at 0 minutes then at 30 minutes, immediately triggering the irrigation cycle at each of these times. The other measurements at 15 and 45 minutes reported 100% soil moisture, confirming successful watering.

Since the soil remained above the irrigation threshold during the final measurement at 60 minutes, no additional pump activation occurred despite the elapsed time. This demonstrates that irrigation decisions were based solely on measured soil conditions rather than fixed timing intervals.

8.5 Overall System Evaluation:

The one-hour experimental run successfully validated the operation and integration of the greenhouse sensing and control system under realistic operating conditions.

The environmental control subsystem responded correctly to temperature changes throughout the test. During the two intentional heating events, the measured temperature exceeded the programmed threshold of 30°C, causing the fan to activate automatically. Once the temperature fell below the threshold, the fan switched off without oscillation or unnecessary activation, demonstrating stable threshold-based thermal control.

The irrigation subsystem also behaved as designed. At system startup, the initial soil moisture reading of 0% triggered an irrigation cycle, which continued for the programmed maximum duration of 12 s before stopping automatically. Later in the experiment, a second low-moisture reading initiated another irrigation cycle. This time, irrigation terminated when the reservoir weight dropped below the minimum allowable threshold of 278.3 g, preventing further depletion of the water supply. These two events experimentally verified both independent pump shutdown mechanisms implemented in the firmware.

The flow sensor detected water movement only while the pump was operating, with no false flow measurements recorded during idle periods. The measured flow events corresponded directly with the observed decreases in reservoir weight and the subsequent increase in soil moisture, demonstrating consistent operation across the sensing and actuation subsystems.

Throughout the experiment, the load cell, flow sensor, DHT11, LDR, and soil moisture sensor operated reliably, and no unintended actuator activations, communication errors, or unstable control behavior were observed. The recorded data therefore confirm that the hardware design, firmware architecture, signal-processing methods, and control algorithms function together as intended, providing reliable automated monitoring and irrigation for the smart greenhouse prototype.

9. Measurement Uncertainty and Error Analysis

We obtained the results of each sensor for a span of 1 hour, and we varied the conditions multiple times during that interval to get different readings.

greenhouse_run1.csv										
	A	B	C	D	E	F	G	H	I	J
	index	time_s	temp_C	hum_pct	weight_g	flow_Lmin	lux	soil_pct	pwm3	pwm6
1	index	time_s	temp_C	hum_pct	weight_g	flow_Lmin	lux	soil_pct	pwm3	pwm6
2	1	1.2	27.1	75.0	-999.0	0.000	573.0	0.5	0	0
3	2	2.2	27.1	75.0	621.4	0.000	542.0	0.3	0	0
4	3	3.2	27.1	75.0	621.4	0.000	545.0	0.3	150	0
5	4	4.2	27.1	75.0	600.2	0.000	524.0	-999.0	180	0
6	5	5.2	27.1	75.0	600.2	0.471	570.0	-999.0	180	0
7	6	6.2	27.1	75.0	557.6	1.255	568.0	-999.0	180	0
8	7	7.2	27.1	75.0	557.6	1.412	569.0	-999.0	180	0
9	8	8.2	27.1	75.0	519.5	1.412	560.0	-999.0	180	0
10	9	9.2	27.1	75.0	519.5	1.412	578.0	-999.0	180	0
11	10	10.2	27.1	75.0	476.4	1.255	579.0	-999.0	180	0
12	11	11.2	27.1	75.0	476.4	1.255	567.0	-999.0	180	0
13	12	12.2	27.1	75.0	432.7	1.255	568.0	-999.0	180	0
14	13	13.2	27.1	75.0	432.7	1.255	570.0	-999.0	180	0
15	14	14.2	27.1	75.0	396.8	1.255	570.0	-999.0	180	0
16	15	15.2	27.6	77.0	396.8	1.255	571.0	-999.0	0	0
17	16	16.2	27.6	77.0	391.0	0.627	575.0	-999.0	0	0
18	17	17.2	27.6	76.0	391.0	0.000	578.0	-999.0	0	0
19	18	18.2	27.6	76.0	391.0	0.000	523.0	-999.0	0	0
20	19	19.2	27.6	75.0	391.0	0.000	513.0	-999.0	0	0
21	20	20.2	27.6	75.0	390.9	0.000	513.0	-999.0	0	0
22	21	21.2	27.6	75.0	390.9	0.000	512.0	-999.0	0	0
23	22	22.2	27.6	75.0	391.0	0.000	584.0	-999.0	0	0
24	23	23.2	27.6	75.0	391.0	0.000	575.0	-999.0	0	0
25	24	24.2	27.6	75.0	390.0	0.000	507.0	-999.0	0	0

We then imported all the sensors data to MATLAB.

The following test cases outline the chronological disturbances applied to the system and the corresponding actuator responses:

- **$t = 0$ min (Initial Start-Up / Dry State):** Ambient temperature was strictly below 30°C. The soil moisture sensor was exposed to ambient air, simulating completely dehydrated soil.

System Response: The Arduino successfully identified the critical moisture deficit and actuated the irrigation pump for the programmed 12-second cycle. The cooling fan remained inactive.

- **$t = 15$ min (Simulated Saturation):**

Condition: The moisture sensor was submerged in water, simulating a post-irrigation saturated state.

System Response: The control logic correctly read the high moisture levels and maintained the pump in the OFF state, preventing resource waste.

- **$t = 30$ min (Dry State with Hardware Fault):**

Condition: The moisture sensor was removed from the water to re-trigger the irrigation cycle. However, the physical reservoir ran out of water during the cycle.

System Response: The pump engaged as expected for the dry state but deactivated prematurely due to the physical constraint of the empty tank, representing a real-world mechanical fault edge-case.

- **$t = 32$ min (Thermal and Luminous Disturbance - "Sunny Weather" Model):**

Condition: Direct body heat (finger) was applied to the temperature sensor to force a rapid thermal spike above the 30°C threshold. Simultaneously, concentrated light was applied to the photoresistor to model intense solar radiation.

System Response: The microcontroller instantly registered the thermal breach and activated the convection cooling fan to mitigate the localized heat spike.

- **$t = 52$ min (Repeated Disturbance Verification):**

Condition: The thermal and luminous disturbances from $t = 32$ were replicated.

System Response: The system demonstrated reliable repeatability, successfully re-engaging the cooling fan as the temperature once again crossed the critical upper limit.

9.1 Equipment Uncertainty (Type B)

Due to the dynamic nature of the testing environment, inherent sensor uncertainty was determined utilizing manufacturer-provided calibration data (Type B). The maximum expected measurement deviations are as follows:

- Temperature: $\pm 2^{\circ}\text{C}$ (Operating range: 0°C to 50°C)
- Flow Rate: $\pm 10\%$ (Operating range: 1 to 30 L/min)
- HX711 + 5kg Load Cell: Combined Error: $\pm 0.05\%$ of Full Scale

These are obtained from the manufacturer data sheets.

9.2 Control System Error and Transient Response

System performance was evaluated by calculating the dynamic error during transient thermal events. When the system was subjected to an artificial thermal spike at $t = 32$ min, the convection cooling loop demonstrated the following response characteristics:

- **Activation Threshold:** 30.0°C
- **Maximum Overshoot:** 2.4°C (from the right most table)
- **Recovery Time:** 257.9 s

A index	B time_s	C temp_C	D hum_pct
NUMBER	NUMBER	NUMBER	NUMBER
1933	1933.2	29.0	92.0
1934	1934.2	29.0	92.0
1935	1935.2	29.0	93.0
1936	1936.2	29.0	93.0
1937	1937.2	29.6	95.0
1938	1938.2	29.6	95.0
1939	1939.2	29.6	96.0
1940	1940.2	29.6	96.0
1941	1941.2	29.7	98.0
1942	1942.2	29.7	98.0
1943	1943.2	30.0	98.0
1944	1944.2	30.0	98.0
1945	1945.2	30.0	98.0
1946	1946.2	30.0	98.0

A index	B time_s	C temp_C
NUMBER	NUMBER	NUMBER
2200	2200.2	30.7
2201	2201.2	30.0
2202	2202.2	30.0
2203	2203.2	30.0
2204	2204.2	30.0
2205	2205.2	30.0
2206	2206.2	30.0
2207	2207.2	30.0
2208	2208.2	30.0
2209	2209.2	29.6

A index	B time_s	C temp_C
NUMBER	NUMBER	NUMBER
2125	2125.2	32.4
2126	2126.2	32.4
2127	2127.2	32.4
2128	2128.2	32.4
2129	2129.2	32.4
2130	2130.2	32.4
2131	2131.2	31.8

These 2 figures show the time when the temperature rose to 30 then continued to increase, and forced back below 30 by the action of the fan.

Then the recovery time is calculated as follows:

$$t_{T=30 \text{ degrees, dropping}} - t_{T=30 \text{ degrees, rising}} = 2201.2 - 1943.3 = 257.9 \text{ sec.}$$

The same process is repeated for the minute 52, when we did the same procedure.

- **Activation Threshold:** 30°C
- **Maximum Overshoot:** 1.2°C (from the right most table)
- **Recovery Time:** 202 s

	A	B	C
	index	time_s	temp_C
	NUMBER	NUMBER	NUMBER
3167	3166	3166.2	29.0
3168	3167	3167.2	29.0
3169	3168	3168.2	29.0
3170	3169	3169.2	29.1
3171	3170	3170.2	29.1
3172	3171	3171.2	29.6
3173	3172	3172.2	29.6
3174	3173	3173.2	29.6
3175	3174	3174.2	29.6
3176	3175	3175.2	30.0

	A	B	C
	index	time_s	temp_C
	NUMBER	NUMBER	NUMBER
3371	3370	3370.2	30.7
3372	3371	3371.2	30.7
3373	3372	3372.2	30.7
3374	3373	3373.2	30.7
3375	3374	3374.2	30.7
3376	3375	3375.2	30.7
3377	3376	3376.2	30.7
3378	3377	3377.2	30.0
3379	3378	3378.2	30.0
3380	3379	3379.2	30.0

A	B	C
index	time_s	temp_C
NUMBER	NUMBER	NUMBER
3190	3190.2	30.7
3191	3191.2	30.7
3192	3192.2	30.7
3193	3193.2	31.1
3194	3194.2	31.1
3195	3195.2	31.2
3196	3196.2	31.2
3197	3197.2	31.2
3198	3198.2	31.2
3199	3199.2	31.2

10. Engineering Constraints Discussion

Designing and implementing the smart greenhouse prototype required balancing several realistic engineering constraints, spanning technical, economic, and environmental factors.

10.1 Technical & Hardware Constraints:

Microcontroller Processing and Memory Limits: The ATmega328P on the Arduino Uno features only 32KB of Flash memory and 2KB of SRAM. Running five distinct sensor drivers, driving local I2C LCD pages, and managing sequential serial parsing pushed the limits of the SRAM. To avoid memory crashes, string literals used in the CSV telemetry stream were wrapped in the macro to save memory.

Sensor Bandwidth and Resolution: The DHT11 introduces a strict sampling constraint, requiring at least 1 to 2 seconds between reads. This prevents high-frequency temperature tracking. Additionally, its 1°C temperature resolution and $\pm 5\%$ humidity accuracy limit its use to broad environmental tracking rather than scientific-grade climate control.

Interrupt Contention: The YF-S201 flow sensor requires precise hardware interrupts to count Hall-effect pulses accurately. Because the Arduino Uno only has two external interrupt pins (D2 and D3), the irrigation pump's PWM control had to be carefully mapped around these pins to avoid timing conflicts during high-speed data logging.

10.2 Sustainability & Environmental Constraints:

Probe Degradation (Galvanic Corrosion): Soil moisture probes operating under a continuous DC voltage degrade rapidly due to electrolysis. To resolve this environmental and hardware lifetime constraint, a **power-gating topology** was implemented, restricting sensor power to a 20ms window per sample. This design choice significantly extends the system's operational lifespan and reduces electronic waste.

10.3 Economic Constraints:

Budget Optimization: The component selection prioritized high cost-efficiency. Using an affordable LDR voltage divider instead of an expensive, dedicated digital lux sensor kept the Bill of Materials (BOM) minimal while still meeting the project's core requirements.

II. Conclusion and Future Work

II.I Conclusion:

The prototype successfully demonstrates a highly functional, multi-modal smart greenhouse system tailored to modern agricultural engineering principles. By integrating diverse sensor types, including ambient climate tracking, reservoir mass calculation, fluid flow measurement, and soil saturation analysis, the system establishes a reliable, real-time edge-monitoring architecture. The project effectively navigates critical hardware-level tradeoffs by executing complex monitoring and automated control loops within the strict memory and processing boundaries of the primary microcontroller.

A major success of the design lies in its targeted hardware mitigation strategies. The implementation of a digital power-gating topology successfully addresses the severe threat of galvanic corrosion on the resistive soil probe, validating a cost-effective approach to extending sensor longevity in high-moisture environments. Furthermore, the integration of logic-level MOSFETs alongside flyback diodes ensures robust electrical isolation for high-current actuators, protecting the system logic from inductive feedback during routine cooling and irrigation operations. Ultimately, this architecture proves that precise, responsive micro-climate management can be achieved using an optimized, accessible hardware stack.

11.2 Future Work:

While the prototype proved successful, several upgrades could elevate the design for commercial deployment:

IoT Integration and Wireless Telemetry: Replacing the Arduino Uno with an ESP32 or adding an ESP8266 Wi-Fi module would allow the system to upload telemetry data to a cloud dashboard (e.g., Blynk, ThingsSpeak) instead of relying on a wired serial USB connection.

Advanced Actuator Control: Upgrading from basic On/Off or static PWM control to a Proportional-Integral-Derivative (PID) control algorithm would allow the cooling fan to adjust its speed dynamically based on how far the actual temperature is from the target, saving energy.

Alternative Soil Sensing Modalities: Swapping out the resistive soil probe for a capacitive soil moisture sensor would eliminate electrode corrosion entirely, making the system even more reliable over long periods.

12. References

Component Datasheets:

- DHT11 datasheet
- HX711 datasheet
- YF-S201 datasheet
- IRLZ44N datasheet
- LM2596 datasheet
- HW-080 soil hygrometer datasheet

Library Documentation/Repositories:

- LiquidCrystal_I2C - markub3327/LiquidCrystal_I2C (fork of johnrickman/LiquidCrystal_I2C), GitHub
- DHT sensor library - Adafruit, GitHub
- HX711 library - bogde/HX711, GitHub